Business Data Mining Homework Session 1

Question 1: Consult the dataset utaut_data.csv. This data was collected as part of a psychometric analysis investigating whether our customers will or will not buy our produce and why. The outcome is Behavioral Intention (B_INT).

(a)Determine how many dimensions are in the data.

CODE:

```
# load data
library(readr)
utaut_data <- read_csv("/Users /desktop/ utaut_data.csv")

# check the dimensions of the data
dim(utaut_data)
```

Data Summary

	Values
Name	utaut_data
Number of rows	216
Number of columns	66
Column type frequency:	
character	1
numeric	65
Group variables	None

Because we need to know the dimensions, we should focus on columns. Columns show dimensions or features. And I think that we can know names of independent variables and dependent variables when looking at names of different columns. Rows show observations. And qualitative column is character type and there are no numbers, so, we don't need to consider this variable. Therefore, the answer is that there is 65 dimensions and 216 observations in a 65-dimensional feature space. Or we can say there are 216 samples and 65 numeric variables. And I think from the qualitative column we can analyze the value of the dependent variable B_Int. For example, if the text mentions good or useful, we can assign it a value of 1. If the text mentions vague or other negative words, we can assign it a value of 0.
I also notice there are 3 abbreviations for column names, int1, int2, int3. But I am not sure whether they represent the outcome (behavioral intention). If int1,int2, int3 mean outcome, that they are dependent variables, and should not counted as dimensions.

(b) Conduct Principle Components to determine the construct scores for these dimensions. Which dimensions best explain the behavioral intention and explain what this might mean.

There are some methods to calculate construct scores: using principal components analysis method, regression method, and simple average or weighted average method. I do it by using PCA method.

CODE:

```
# Step 1: find the numeric columns
str(utaut_data)

# Step 2: choose numeric columns and use to do PCA
X_numeric <- utaut_data[, sapply(utaut_data, is.numeric)]

# Step 3: standardize the numeric data
X_scaled <- scale(X_numeric)

# Step 4: PCA
pca <- prcomp(X_scaled, center = TRUE, scale. = TRUE)

# Step 5: find the proportion of variance explained by each principal component
summary(pca)
```

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9
Standard deviation	4.7957	2.9385	1.77243	1.6166	1.41233	1.36664	1.32047	1.26293	1.19120
Proportion of Variance	0.3538	0.1328	0.04833	0.0402	0.03069	0.02873	0.02683	0.02454	0.02183
Cumulative Proportion	0.3538	0.4867	0.53501	0.5752	0.60590	0.63463	0.66146	0.68599	0.70783

I think a cumulative proportion of 70% is sufficient to represent the original variables, so I choose the first 9 principle components.

```
# Step 6: find the loading matrix
loadings <- pca$rotation
head(loadings[, 1:5])
# choose first 5 components
```

	PC1	PC2	PC3	PC4	PC5
ANX1	0.11408988	-0.14654732	-0.002269338	0.02464023	0.19106682
ANX2	0.08849050	-0.13662415	0.024566589	0.10558548	0.10231594
ANX3	0.08374241	-0.17384394	0.024367918	0.05707956	0.19664986
ANX4	0.08425803	-0.21243798	0.087923690	-0.02892807	0.21465570
ATTU1	-0.15949610	0.02129524	0.161512814	0.09408033	-0.02457279
ATTU2	-0.14672989	0.01290602	0.149834858	0.08091522	0.02543696

```
# Step 7: find the construct scores
construct_scores <- as.data.frame(pca$x)
head(construct_scores)
```

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9
1	-6.8231	-0.3312	-1.9841	-1.1810	0.9591	-1.5262	1.7984	-0.3504	-1.2392
2	3.7916	-1.0466	-0.7908	0.3997	1.0455	-0.1392	-3.7525	-1.7044	0.8887
3	-1.2936	-2.7715	0.8258	-1.0257	-0.4959	-0.1683	0.1816	1.3648	-0.7166
4	1.2674	0.8484	0.6080	0.3321	0.8592	0.5391	-0.3894	0.3947	1.4684
5	-1.7515	-1.3898	0.5456	-0.6260	0.4143	-0.8575	-1.2138	0.5337	0.2191
6	1.2311	-0.5396	2.1861	0.0306	2.1922	0.0699	-0.4245	0.2242	-1.2533

(there are 6 rows and 65 columns.)

I think we should select the first 9 principal components and perform correlation or regression analysis on the construct scores and behavioral intention (B_Int) dependent variables to determine which principal components are significantly associated with B_Int. because we cannot directly use the construct score to get any conclusions. But now I don't have the figures of dependent variable B_Int.

Based on the principal component loading matrix (step 6), we might explain the meaning of each significant principal component (which original variables contribute the most) and then discuss why these dimensions can explain behavioral intention. The first two principal components (PC1 and PC2) best explain the behavioral intention. PC1 and PC2 together explain 48.67% of the total variance, meaning they capture most of the structure in the data. PC1 explains a large portion of variance (35.38%) and PC2 explains 13.28%. From the first 5 components, we find that ANX1-4 are positive on PC1 and are negative on PC2, and ATTU1-2 are negative on PC1, are small positive on PC3.

(c) What are your thoughts and insights on using Principle Components in survey research?

The principal components are uncorrelated and have decreasing variance. The sum of the variances of the principal components we choose should account for the majority of the total variance. I think more than 70% is sufficient. Using principal components helps reduce dimensions.

But as mentioned above, I think using PCA alone is not enough. We need to do regression analysis on the construct scores of the principal components and the dependent variables to determine which principal components are significantly related to behavioral intention. And use the principal component loading matrix to explain which original variables contribute the most.

Question 2.9: Normalise the data in Table 2.17, showing calculations.

There are several methods to normalize the data. Following are the 2 different methods, one is min-max normalization, and the other is z-score normalization.

Method1: Min-Max Normalisation

- Original: age between 25 and 65, income between 39000 and 192000
- Normalisation: Between 0 and 1

$$\text{Formula} = \frac{(\text{x} - min_x)}{(max_x - min_x)}$$

CODE:

```
# create a  function to do min-max normalization
min_max_normalize <- function(x) {return ((x - min(x)) / (max(x) - min(x)))}

# apply the function to the data
age <- c(25, 56, 65, 32, 41, 49)
income <- c(49000, 156000, 99000, 192000, 39000, 57000)
normalized_age <- min_max_normalize(age)
normalized_income <- min_max_normalize(income)

# combine the normalized data into a data frame
data.frame(Age = age, Normalized_age= normalized_age, Income = income, Normalized_income=normalized_income)
```

Age	Normalized	Income	Normalized_income
25	0.000	49000	0.06535948
35	0.775	156000	0.76470588
65	1.000	99000	0.39215686
32	0.175	192000	1.00000000
41	0.400	39000	0.00000000
49	0.600	57000	0.11764706

Method2: Z-Score Normalisation

- Mean value μ become 0 and standard deviation σ become 1

$$Z = \frac{(X - \mu)}{\sigma}$$

CODE:

```
# create a function to do Z-Score Normalisation
z_score_normalize <- function(x) {return ((x - mean(x)) / sd(x))}

# apply the function to the data
normalized_age_z <- z_score_normalize(age)
normalized_income_z <- z_score_normalize(income)

# combine the normalized data into a data frame
data.frame(Age = age, Z_Normalized_age= normalized_age_z, Income = income, Z_Normalized_income=normalized_income_z)
```

Age	Z_Normalized_age	Income	Z-Normalized_income
25	-1.3132530	49000	-0.790026927
56	0.7567898	156000	0.911977392
65	1.3577700	99000	0.005302194
32	-0.8458239	192000	1.484614360
41	-0.2448438	39000	-0.949092751
49	0.2893608	57000	-0.662774268

Both methods rely on all the data in the sample. The drawback is that when new data is added, the min and max values will change and need to be redefined, otherwise the effect will be affected.

Question 2.8: Can normalizing the data change which two records are farthest from each other in terms of Euclidean distance?

Yes, normalizing the data can change which two records are farthest apart in terms of Euclidean distance.
The Euclidean distance between two records $\mathrm{x} = (x_1, x_2, \dots, x_n)$ and $\mathrm{y} = (y_1, y_2, \dots, y_n)$ is

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

This measure depends directly on the scale of each attribute. If one attribute has much larger numeric values or variance, it dominates the distance measure.

Question 4: Chemical Features of Wine. Table 4.13 shows the PCA output on data (non-normalized) in which the variables represent chemical characteristics of wine, and each case is a different wine.

(a) The data are in the file Wine.csv. Consider the rows labeled "Proportion of Variance." Explain why the value for PC1 is so much greater than that of any other column.

There are two reasons in which PC1 can be much greater than that of any other columns in the non-normalized case. In non-normalized PCA, the eigenvalues of the principal components represent the variance along that direction. The eigenvalue of PC1 is equal to the maximum variance along a linear combination of the data. If the measurement scale or variance of some original variables is large, they will dominate the covariance matrix, resulting in very large eigenvalues of PC1.

Another possible reason is strong intercorrelations among many variables. When a set of variables move together in the same "direction," their shared variance is captured by PC1, making PC1 large. If we observe that many variables have loadings on PC1 with the same sign and large magnitude, this indicates the presence of a "common factor," causing PC1 to capture a significant amount of shared variance.

CODE:

```
# 1. Load the data
library(readr)
wine<- read_csv("/Users/nisa/desktop/Wine.csv.gz")

# 1Check the structure of the data (to see if there are non-numeric columns)
str(wine)
head(wine)

# 2. Check variance of each variable (for non-normalized PCA)
vars <- apply(wine_num, 2, var, na.rm = TRUE)
print(vars)            # Variance of each column
round(vars, 4)

# 3. Compute correlation matrix to see relationships among variables
cor_mat <- cor(wine_num, use = "pairwise.complete.obs")

# View a subset of the correlation matrix
print(round(cor_mat[1:6, 1:6], 3))

# Count how many variable pairs have a strong correlation
high_corr_pairs <- sum(abs(cor_mat[upper.tri(cor_mat)]) > 0.7)
cat("Number of variable pairs with |corr| > 0.7:", high_corr_pairs, "\n")

# 4. Perform PCA on non-normalized data (based on covariance matrix)
pca_cov <- prcomp(wine_num, center = TRUE, scale. = FALSE)

# 4.1 Output standard deviations, eigenvalues, and proportion of variance
sdev <- pca_cov$sdev
eigenvalues <- sdev^2
propVar <- eigenvalues / sum(eigenvalues)
res_table <- data.frame(
        StdDev = round(sdev, 4),
        Eigenvalue = round(eigenvalues, 4),
         PropVar = round(propVar, 4),
        CumVar = round(cumsum(propVar), 4)
)
print(res_table)

# 4.2 View the loadings matrix (variable coefficients for each PC)
```

```
loadings <- pca_cov$rotation   # columns = PCs
round(loadings[, 1:5], 3)      # Show first 5 PCs
```

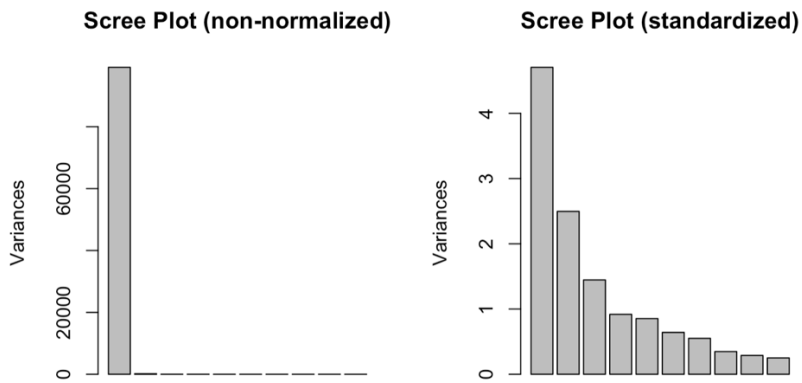
```
# 4.3 Identify variables contributing most to PC1 (by absolute loadings)
abs_loadings_PC1 <- abs(loadings[, 1])
sort(abs_loadings_PC1, decreasing = TRUE)
```

PC	StdDev <dbl>	Eigenvalue <dbl>	PropVar <dbl>	CumVar <dbl>
PC1	314.9632	99201.7895	0.9981	0.9981
PC2	13.1353	172.5353	0.0017	0.9998
PC3	3.0722	9.4381	0.0001	0.9999
PC4	2.2341	4.9912	0.0001	1.0000
PC5	1.1085	1.2288	0.0000	1.0000
PC6	0.9171	0.8411	0.0000	1.0000
PC7	0.5282	0.2790	0.0000	1.0000
PC8	0.3891	0.1514	0.0000	1.0000
PC9	0.3348	0.1121	0.0000	1.0000
PC10	0.2678	0.0717	0.0000	1.0000
PC11	0.1938	0.0376	0.0000	1.0000
PC12	0.1452	0.0211	0.0000	1.0000
PC13	0.0906	0.0082	0.0000	1.0000

PC1 explains 99.8% of the total variance, which is extremely high. This means nearly all variation in the raw data is captured by one single direction. Such dominance usually happens when one or more variables have much larger numerical scales or variances than others. The remaining components (PC2, PC3, etc.) contribute almost nothing (less than 0.2% in total), which shows that PCA in this form mainly reflects not true structure in the data.

```
# 5. Plot Scree Plot (compare covariance-based and correlation-based PCA)
# Non-normalized (covariance PCA)
par(mfrow = c(1, 2))
plot(pca_cov, main = "Scree Plot (Covariance PCA, Non-normalized)")
```

```
# 6. Perform PCA on standardized data (based on correlation matrix)
pca_cor <- prcomp(wine_num, center = TRUE, scale. = TRUE)
plot(pca_cor, main = "Scree Plot (Standardized)")
par(mfrow = c(1, 1))
```



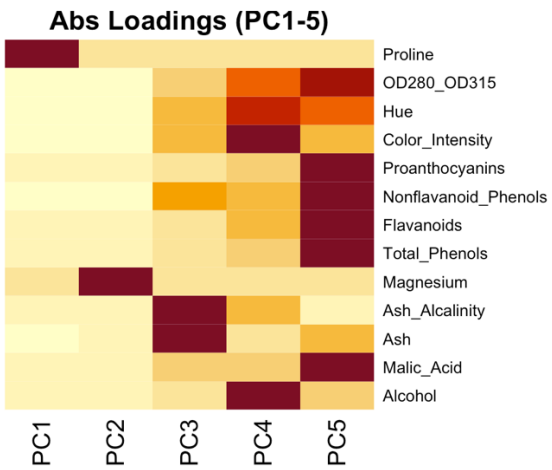

```
# 6.1 Compare explained variance between the two methods
eig_cov <- (pca_cov$sdev)^2
eig_cor <- (pca_cor$sdev)^2
prop_cov <- eig_cov / sum(eig_cov)
prop_cor <- eig_cor / sum(eig_cor)
compare <- data.frame(
        PC = paste0("PC", 1:length(prop_cov)),
```

```
            PropVar_Cov = round(prop_cov, 4),
           PropVar_Cor = round(prop_cor, 4)
)
print(compare[1:10, ])
```

PC <chr>	PropVar_Cov <dbl>	PropVar_Cor <dbl>
PC1	0.9981	0.3620
PC2	0.0017	0.1921
PC3	0.0001	0.1112
PC4	0.0001	0.0707
PC5	0.0000	0.0656
PC6	0.0000	0.0494
PC7	0.0000	0.0424
PC8	0.0000	0.0268
PC9	0.0000	0.0222
PC10	0.0000	0.0193

After standardization, PC1 explains 36.20% of the total variance, PC2 explains 19.21%, etc. This shows a much more balanced distribution of explained variance across components. It means that after controlling for scale, there are multiple underlying chemical dimensions that explain the variation in wine composition. Now PCA reflects correlation patterns among variables.

```
# 7. Visualize loadings as a heatmap
library(graphics)
# Simple heatmap for absolute loadings (first 5 PCs)
heatmap(abs(loadings[, 1:5]), Colv = NA, Rowv = NA, main = "Abs Loadings (PC1–PC5)")
```



(b) Comment on the use of normalization (standardization) in part (a).

I think that non-normalized PCA shows where the absolute variance is largest and normalized PCA shows where the strongest relationships are. After normalization, each variable is transformed to have a mean of 0 and a standard deviation of 1, so all variables contribute equally to the analysis.